CloudAI AWS step-by-step guide

Updated 12 October 2026 for the separate team review project.

1. What changes after the model trials?

AWS is a computer we rent from Amazon. SageMaker lets us run a notebook on it. Our job is to train and tune models there, save the real results, and explain them.

Uploading BESTIJA's saved model only demonstrates upload. It does not demonstrate AWS training. The assignment asks for cloud training and tuning and the downloaded SageMaker notebooks. Plan one run per dataset unless Jochen confirms one overall is enough. Course assignment

Dataset	First AWS route	What it needs
Mushrooms	Extra Trees, four settings	CPU, search rows and separate calibration rows
Citi Bike	SVR/Hist weather blend, four settings	CPU, daily counts and archived forecasts
Citi Bike optional	GRU/CatBoost, two settings	Allowed CUDA GPU, expanded 2022/2023 counts and forecasts

The local weather GRU blend has the lowest recorded bike MAE: 13,263 rides/day. Its CPU alternative is close at 13,494 and has a better RMSE. The apparent gain over XGBoost is uncertain. Use the CPU route first if the Sandbox has no GPU.

Extra Trees' cautious rule caught 300/303 poisonous rows but flagged 474/497 edible rows. It still missed three poisonous examples. Do not describe that as 99% accuracy or a safe classifier. The old threshold belongs to the old fitted model. The AWS runner calibrates the newly fitted model separately.

Final tests remain unused. Cloud runs use training CV and mushroom calibration. They do not reopen the 800-row mushroom check or the July-September bike comparison.

2. Open the university AWS account

1. Open university Canvas, the Deep Learning course, then **AWS Canvas**.

2. Open AWS Academy **Modules > Sandbox > Sandbox environment**. Review Vocareum's terms yourself. AWS Academy modules

3. Read the Sandbox instructions. Record the budget, allowed region, execution role, allowed instance types and what survives **End Lab**. Those account details are still unverified.

4. Select **Start Lab**. Wait for the ready status. Open **AWS**.

5. In the allowed region, open **Amazon SageMaker AI > Notebook instances > Create notebook instance**.

6. Name it `cloudai-training`. Select an allowed small CPU type, such as `ml.t3.medium` if permitted. Use the course's existing IAM execution role. Select `notebook-al2023-v1` / JupyterLab 4. Use the course's storage allowance; package installation may need more than the default 5 GB.

7. Wait for **InService**, then open **JupyterLab**.

AWS documents this notebook route and AL2023. Its standard Python kernel is 3.10, so we register a separate 3.12.10 kernel. Create a notebook Â· AL2023 kernels

If the account denies the notebook, role or region, keep the error and ask Jochen for the permitted route. Our provenance check expects a notebook instance. Studio and SageMaker Canvas need different setup. The Sandbox and SageMaker Canvas are different products.

3. Upload this review copy

Before the team pushes this version, use the prepared ZIP. After all six handoff commits are pushed, clone that exact GitHub commit instead.

1. In the local review folder, find `CloudAI_clean_project.zip`.

2. In the AWS JupyterLab file browser, open `/home/ec2-user/SageMaker` and upload that ZIP.

3. Open **File > New > Terminal**. Run:

```
cd /home/ec2-user/SageMaker
python3 -m zipfile -e CloudAI_clean_project.zip .
cd CloudAI_clean_project
python3 scripts/check_package.py
```

The ZIP expands into `CloudAI_clean_project/`. The package check verifies its recorded source, model and small data files. Do not continue after a mismatch. Raw trip ZIPs and environments are not included. Small daily tables are enough for this cloud run.

After the team later pushes this version, cloning that exact commit can replace the ZIP step. Record the package manifest or actual commit used. Keep results before pulling any later changes.

4. Install the CPU environment

These are Linux commands in the AWS terminal. Run them one at a time. Stop if any line fails.

```
python3 -m pip install --user uv
python3 -m uv python install 3.12.10
python3 -m uv venv .venv-review --python 3.12.10 --seed
python3 -m uv pip install --python .venv-review/bin/python \
  -r requirements-review-linux.lock.txt
.venv-review/bin/python -m pip check
.venv-review/bin/python -m ipykernel install --user \
  --name cloudai-review \
  --display-name "CloudAI review (Python 3.12.10)"
.venv-review/bin/python --version
```

The last line must say Python 3.12.10. Refresh JupyterLab if the kernel is missing. Select **CloudAI review** for both CPU notebooks. uv Python installation Â· uv Jupyter kernels

This Linux lock was resolved locally. Actual AWS installation is still unverified. The fresh Windows CPU environment was checked separately. Neither means the Sandbox supports all resources.

PyTorch is optional for the CPU AWS runs. To display the saved GRU predictions on a CPU, add:

```
python3 -m uv pip install --python .venv-review/bin/python \
  torch==2.11.0 --index-url https://download.pytorch.org/whl/cpu
```

The original team PyCaret notebooks keep their Python 3.11 environment. These new AWS notebooks use the review environment. Do not mix the two.

5. Train and tune both CPU models

1. Open `mushrooms/10_aws_extra_trees.ipynb`.

2. Select the review kernel. Replace `YOUR NAME` with the actual operator's name.

3. Run the setup, training and result cells in order. Save the notebook.

4. Read `run_record.json` and `tuning_results.csv` in the printed run folder. Keep warnings.

5. Repeat with `citibike/16_aws_cpu.ipynb`.

The mushroom run compares four Extra Trees settings with five grouped folds on 2,559 search rows. It fits the chosen setting and uses the separate 640 calibration rows to choose a new cautious threshold. Calibration results are not an independent test. The one quarantined duplicate stays excluded.

The CPU bike run compares four nearby SVR/Hist settings. It trains on 153 targets through 30 June and checks the same three chronological 21-day folds. The fixed 75%/25% blend weights stay unchanged. Preparation is fitted inside each training fold. The search changes SVR strength and histogram iteration count. It does not search thousands of settings again.

Each run writes a trained model, tuning results and a run record in its own folder. The mushroom run also saves calibration predictions. The record contains package versions, source/data hashes and a redacted notebook identifier. AWS supplies the notebook metadata file. AWS notebook metadata

No cloud scores exist until these cells actually run. A similar or worse result is valid. Keep the difference between training CV, calibration, development confirmation and final test clear.

6. Optional: reproduce GRU training

Do this only if the course allows the GPU type and its cost. The exact research GRU fitting code requires CUDA. Saved inference works on CPU.

1. Export any completed CPU work first.

2. Stop the notebook instance. Wait for **Stopped**. Change it to an allowed GPU type using **Update settings**, then start it. An example is `ml.g5.xlarge` only if the Sandbox permits it. Do not assume a GPU is included in your credits. Change instance settings

3. In the AWS terminal, make a separate environment:

```
python3 -m uv venv .venv-gru --python 3.12.10 --seed
python3 -m uv pip install --python .venv-gru/bin/python \
  -r requirements-review-linux.lock.txt
python3 -m uv pip install --python .venv-gru/bin/python \
  torch==2.11.0 --index-url https://download.pytorch.org/whl/cu128
.venv-gru/bin/python -m pip check
.venv-gru/bin/python -m ipykernel install --user \
  --name cloudai-gru --display-name "CloudAI GRU (CUDA)"
nvidia-smi
.venv-gru/bin/python -c "import torch; print(torch.cuda.is_available())"
```

The final check must print `True`. Otherwise use the CPU route and keep the error. PyTorch has separate CPU and CUDA package indexes. uv and PyTorch

4. Open `citibike/17_aws_gru_optional.ipynb`, choose **CloudAI GRU**, enter your name, then run and save.

This uses 518 training targets from the expanded history, the same checking dates, and two GRU sizes. Scaling and epoch selection use only the inner training side. Actual cloud results may differ across GPUs. They must be recorded rather than assumed equal.

7. Save evidence, then stop the machine

1. Save each successfully executed notebook.

2. Run its final export cell. Save again and rerun export so the ZIP includes the latest saved outputs.

3. Download the evidence ZIP from JupyterLab. Open it locally. Check the notebook outputs, `run_record.json`, tuning table and fitted model.

4. Save a console screenshot showing notebook name, instance type, region and status. Hide account identifiers. Do not share credentials.

5. In the notebook's Markdown, add the real operator, date, contribution and a short factual result. Compare the same metric and split.

6. Keep cloud evidence separate from the original local reports. The team can later review and copy it into GitHub. Do not push from the temporary AWS account.

7. In AWS, select **Notebook instances > cloudai-training > Actions > Stop**. Wait for **Stopped**. This ends notebook compute charges. Storage and other resources can still cost money. AWS stop behaviour

8. Follow the Sandbox's cleanup instructions and select **End Lab**. Export before ending the lab; its persistence rules are unverified.

Closing the browser alone does not stop the notebook. Files outside `/home/ec2-user/SageMaker` can be lost when a notebook is stopped. After a restart, recheck the environment and kernel registration. AWS notebook storage

8. Connect the results to our release

AWS training does not finish hosting. We still need the hosted API, webpage, model update pipeline, final tests and submission. The project contains a Dockerfile and workflow; a host and its secret still need configuration. AWS alone does not finish the project.

Use complete fitted pipelines. Do not export only the tree estimator. Preparation, target restoration, blend weights and the mushroom threshold belong with the model. The app's model manifest records versions and hashes. A new mushroom fit needs its own calibrated threshold in that manifest.

The Citi Bike GRU path needs 28 prior daily counts and six prediction-day forecast numbers. The CPU SVR/Hist path also needs those forecasts for the previous seven dates. Counts-only XGBoost needs no forecasts. Never fill absent weather with invented values just to make the API run.

Before the final test, agree the release model, input contract, training boundary and threshold procedure. Freeze them. Then score the 1,000 mushroom test rows and October-December bike days once. Record the results without tuning against them.

Questions for Jochen: one AWS model per dataset or overall? Is notebook training/tuning enough, or are managed jobs required? Which region, role, budget and instances are allowed?

Course deadline: **15 October 2026, 23:59**. Aim to finish on 14 October. Canvas submission

Weather source: Open-Meteo's Previous Runs API, NOAA GFS and JMA GSM, CC BY 4.0. Daily New York aggregation and rain interval correction are our modifications. Exact historical publication times were not verified.